

Университет ИТМО
Факультет программной инженерии и компьютерной техники

Отчет
Лабораторная работа №4
Основы программной инженерии
Вариант 883878

Группа
Выполнили:
Преподаватели

Р3209
Исаева Александра-Ирина Антоновна
Ермаков М.К.
Клименков С.В.

Содержание

1	Задание	2
1.1	Вопросы к защите	2
2	Реализация MBean-классов	3
2.1	Состав разработанных классов	3
3	Мониторинг с помощью JConsole	3
3.1	Подключение	3
3.2	Показания разработанных MBean-классов	3
3.2.1	Счётчик точек	4
3.2.2	Калькулятор площади	5
3.3	Оповещение о двух промахх подряд	5
3.4	Время, прошедшее с момента запуска виртуальной машины	6
4	Мониторинг и профилирование с помощью VisualVM	6
4.1	Подготовка инструмента	6
4.2	Вкладка Monitor и смысл её графиков	7
4.3	Графики изменения показаний MBean во времени	7
4.4	Определение потока с наибольшим потреблением CPU	9
5	Поиск и устранение проблемы производительности в программе HttpUnit	10
5.1	Описание	10
5.2	Воспроизведение	10
5.2.1	Подготовка и запуск	10
5.2.2	Обнаружение симптома на вкладке Monitor	10
5.2.3	Определение типа утекающих объектов	12
5.2.4	Подтверждение по исходному коду	12
5.3	Исправление	12
5.4	Сравнение	12
6	Выводы	13

1 Задание

Для своей программы из лабораторной работы №3 по дисциплине Веб-программирование реализовать:

- MBean, считающий общее число установленных пользователем точек, а также число точек, не попадающих в область. В случае, если пользователь совершил 2 «промаха» подряд, разработанный MBean должен отправлять оповещение об этом событии.
- MBean, определяющий площадь получившейся фигуры.

С помощью утилиты JConsole провести мониторинг программы:

- Снять показания MBean-классов, разработанных в ходе выполнения задания 1.
- Определить время (в мс), прошедшее с момента запуска виртуальной машины.

С помощью утилиты VisualVM провести мониторинг и профилирование программы:

- Снять график изменения показаний MBean-классов, разработанных в ходе выполнения задания 1, с течением времени.
- Определить имя потока, потребляющего наибольший процент времени CPU.

С помощью утилиты VisualVM и профилировщика IDE NetBeans, Eclipse или Idea локализовать и устранить проблемы с производительностью в предложенной программе. По результатам локализации и устранения проблемы необходимо составить отчёт, в котором должна содержаться следующая информация:

- Описание выявленной проблемы.
- Описание путей устранения выявленной проблемы.
- Подробное (со скриншотами) описание алгоритма действий, который позволил выявить и локализовать проблему.

Студент должен обеспечить возможность воспроизведения процесса поиска и локализации проблемы по требованию преподавателя.

1.1 Вопросы к защите

- Мониторинг и профилирование. Основные понятия. Отличия мониторинга от профилирования.
- Инфраструктура для организации мониторинга и профилирования в составе JDK. JMX.
- MBeans. Основные понятия. Архитектура фреймворка.
- Утилита JConsole. Возможности, область применения.
- Утилита Visual VM. Возможности, область применения.
- Удалённый мониторинг и профилирование приложений на платформе Java.

2 Реализация MBean-классов

2.1 Состав разработанных классов

Все разработанные классы размещены в пакете `isaxel.jmx` веб-приложения

<https://github.com/isaxel/itmo/tree/main/4%20software%20engineering/labs/4%20lab>

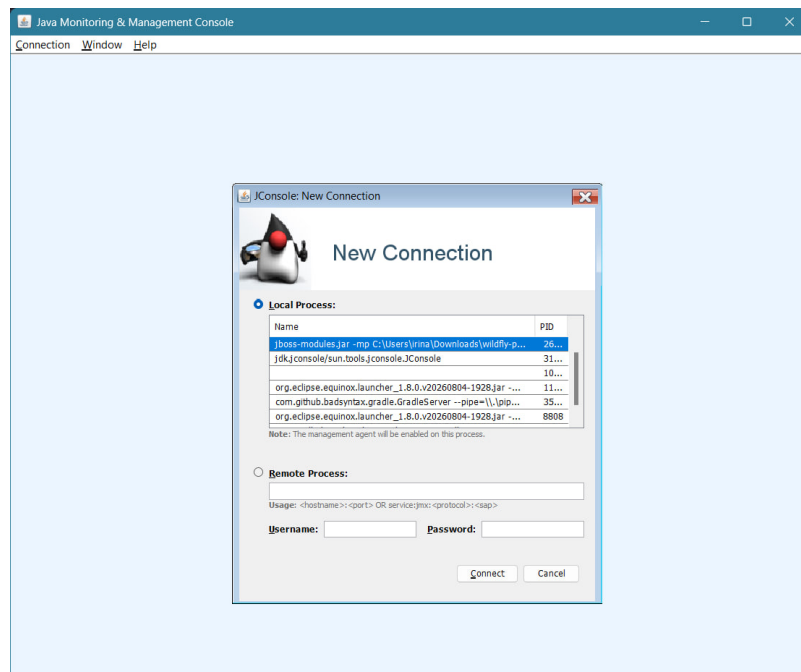
3 Мониторинг с помощью JConsole

3.1 Подключение

Приложение развёрнуто на сервере WildFly Preview 26.1.3. JConsole запущена командой `jconsole` из состава того же JDK, на котором работает сервер.

В стартовом окне New Connection доступны две вкладки: Local Process и Remote Process. Использовалось локальное подключение: в списке процессов выбран

`org.jboss.modules.Main` - главный класс загрузчика модулей WildFly, под которым работает сервер приложений.

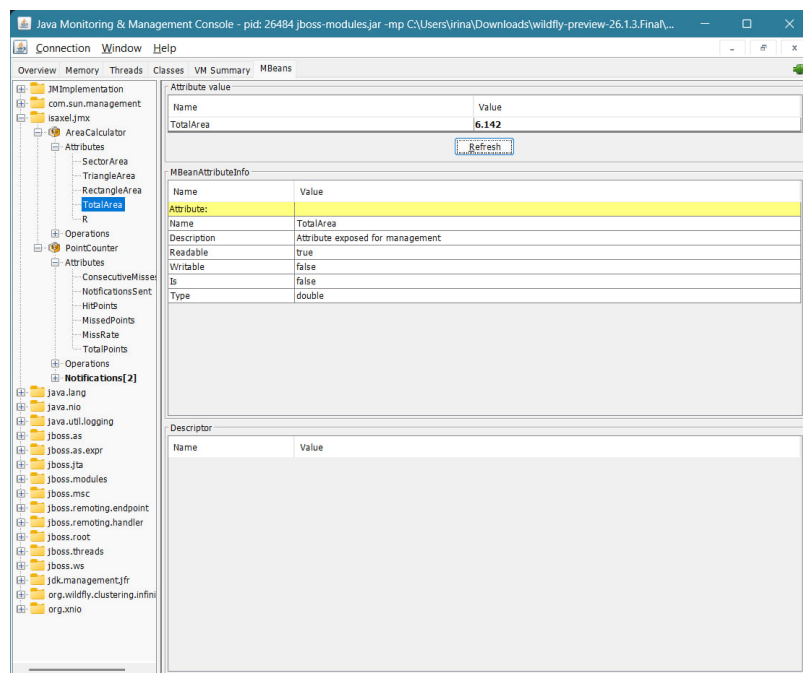
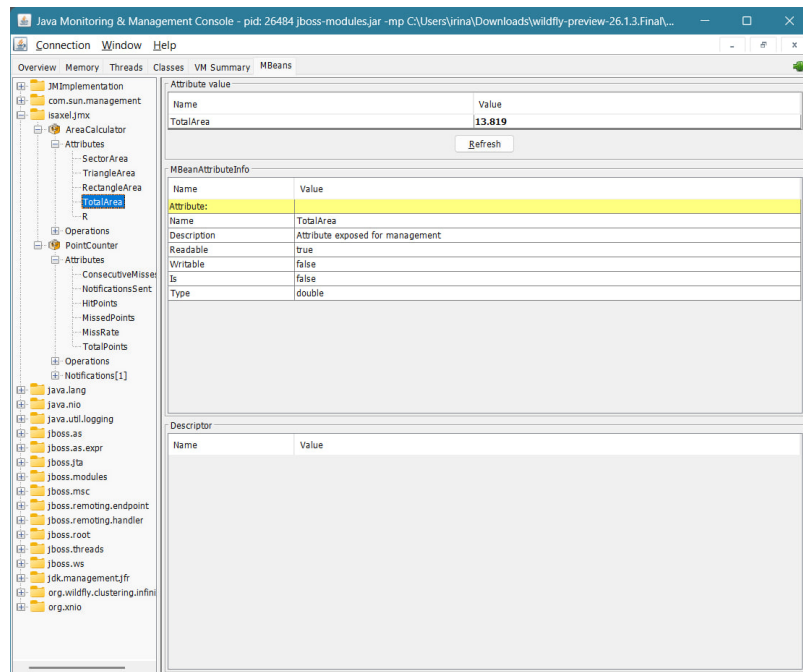


3.2 Показания разработанных MBean-классов

Разработанные бины находятся на вкладке **MBeans**, в узле `isaxel.jmx`. Внутри домена два узла — `PointCounter` и `AreaCalculator`, у каждого подузлы `Attributes`, `Operations` и `Notifications`.

Важная особенность JConsole: значения атрибутов не обновляются самостоятельно. Клиент опрашивает бин по запросу, поэтому после действий в веб-интерфейсе требуется нажать кнопку Refresh.

3.2.2 Калькулятор площади

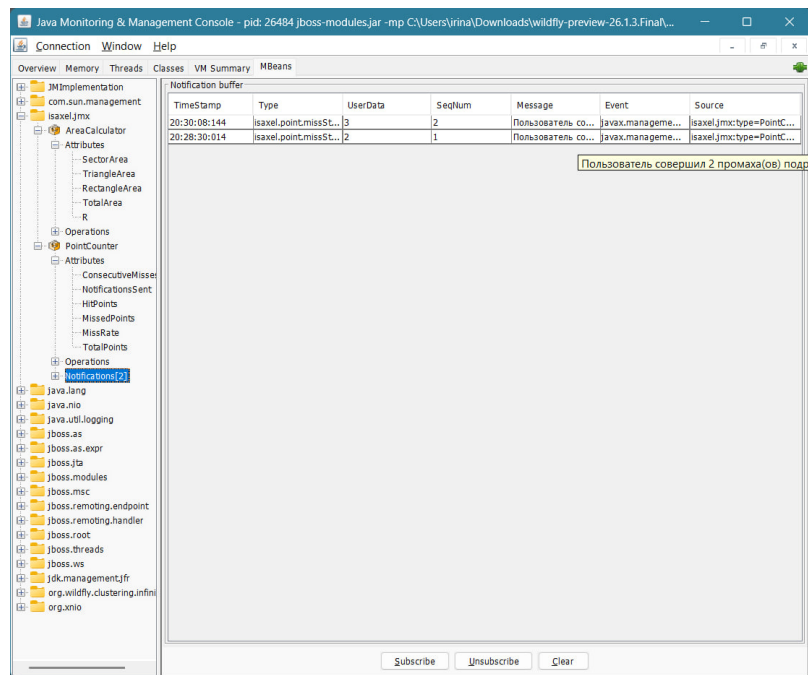


При разных значениях R разная площадь.

3.3 Оповещение о двух промахах подряд

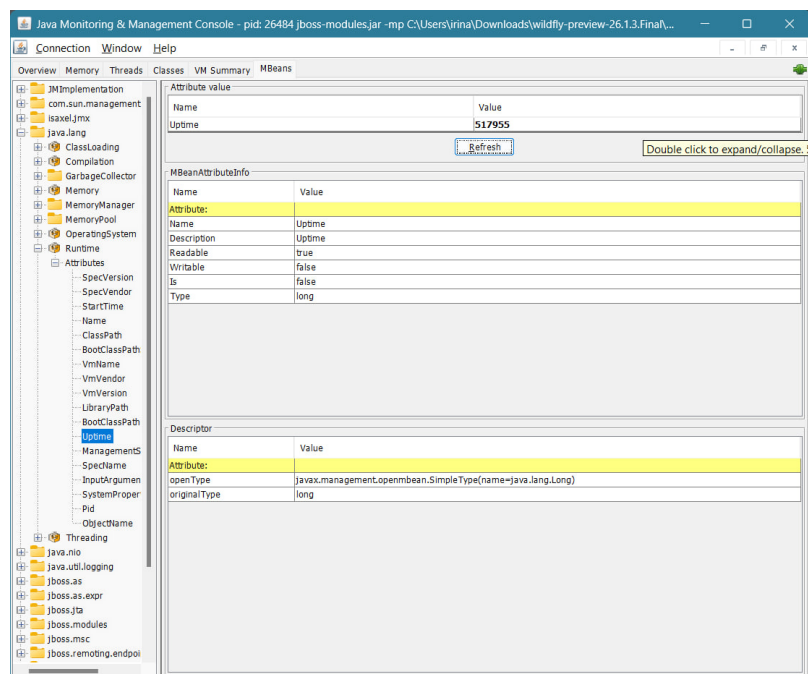
Уведомления находятся на вкладке Notifications узла PointCounter. До подписки список пуст, а рядом с названием вкладки в скобках отображается ноль. После нажатия кнопки Subscribe клиент регистрирует слушателя на сервере и начинает получать события.

В приложении установлены две точки подряд, не попадающие в область. Второй промах вызвал отправку уведомления.



3.4 Время, прошедшее с момента запуска виртуальной машины

Искомая величина предоставляется платформенным бином, а не разработанным в рамках задания. Путь в дереве: **MBeans** → **java.lang** → **Runtime** → **Attributes** → **Uptime**.
Время в миллисекундах



4 Мониторинг и профилирование с помощью VisualVM

4.1 Подготовка инструмента

Использована VisualVM 2.2.2.

В базовой поставке вкладка MBeans отсутствует, поэтому дополнительно установлен плагин VisualVM-MBeans через Tools → Plugins → Available Plugins. Подключение к процессу выполняется через панель Applications, узел Local, двойной щелчок по процессу `org.jboss.modules.Main`.

4.2 Вкладка Monitor и смысл её графиков

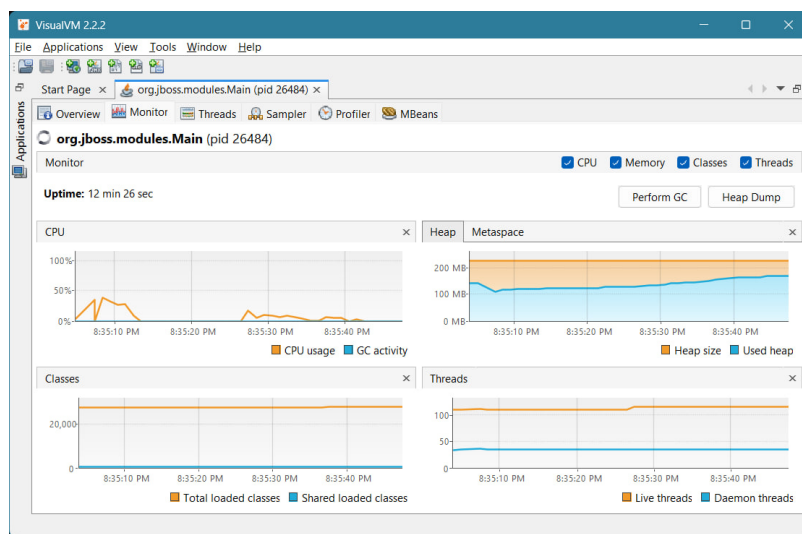
Вкладка Monitor содержит четыре графика, каждый из которых отвечает за свой класс проблем.

График CPU показывает две величины. Линия CPU usage — доля процессорного времени, потребляемая процессом. Линия GC activity — доля времени, которое виртуальная машина тратит на сборку мусора.

График Heap — основной для данной работы. На нём две линии: Size — размер кучи, выделенный виртуальной машиной, и Used — фактически занятая память. Нормальная форма графика Used — пила: память растёт по мере создания объектов и резко падает при срабатывании сборщика мусора.

График Metaspace отражает память, отведённую под метаданные загруженных классов. В норме он растёт на старте приложения и выходит на постоянный уровень.

График Classes и Threads показывает число загруженных классов и живых потоков. Постоянный рост числа потоков означает, что приложение создаёт потоки, но не завершает их.



4.3 Графики изменения показаний MBean во времени

Для построения графика показаний собственного бина используется вкладка MBeans: в дереве выбирается `isaxel.jmx` → `PointCounter`, открывается вкладка Attributes, и двойным щелчком по числовому атрибуту открывается отдельное окно атрибута с вкладкой Chart. С этого момента VisualVM начинает периодически опрашивать значение и строить его во времени. Во время наблюдения в веб-приложении последовательно устанавливались точки.

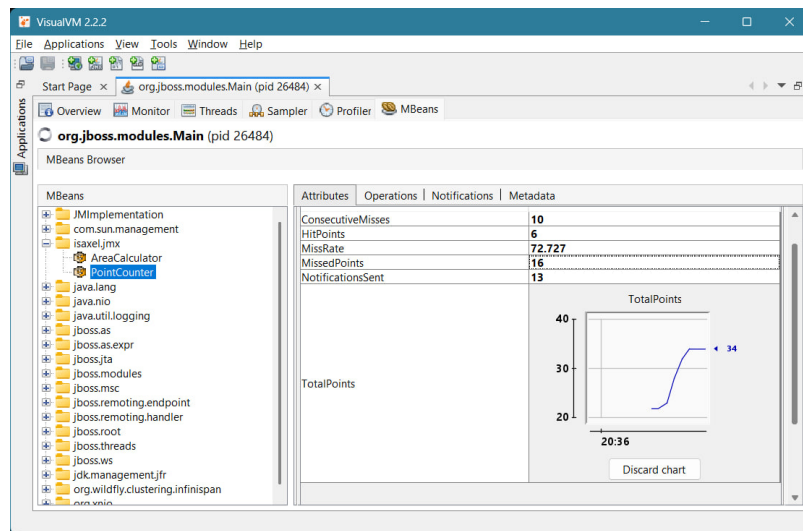


График **TotalPoints** имеет вид ступенчатой линии. Каждая ступень соответствует одной установленной точке, высота всех ступеней одинакова и равна единице — это подтверждает, что каждое действие пользователя регистрируется ровно один раз, без пропусков и без двойного счёта. Горизонтальные участки между ступенями — промежутки, когда пользователь не совершал действий.

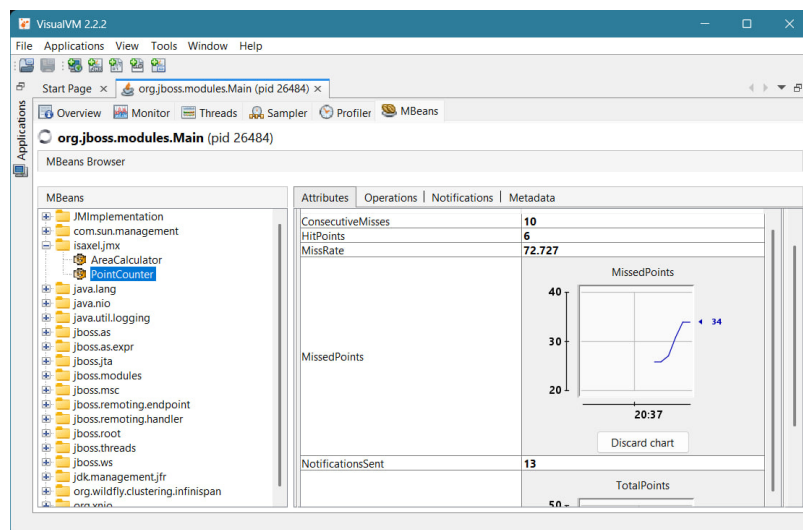


График **MissedPoints** построен в том же масштабе времени и повторяет форму предыдущего, но растёт только в моменты промахов. Он всегда проходит не выше графика **TotalPoints**, поскольку промахи являются подмножеством всех точек.

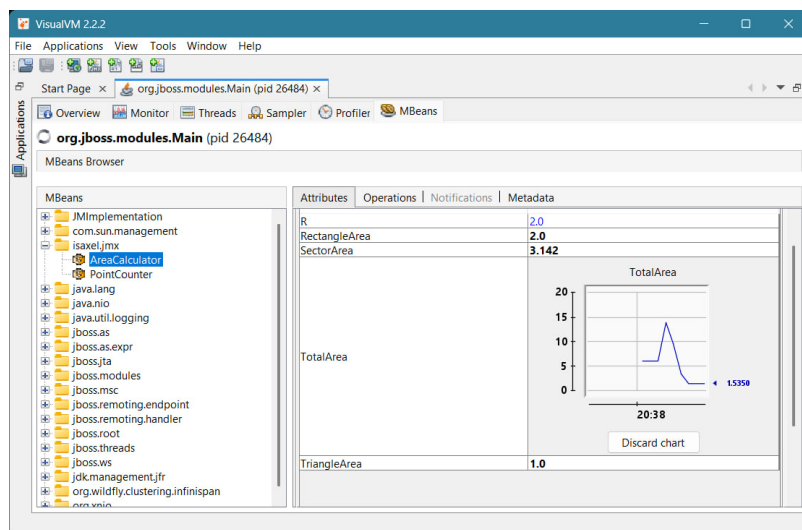
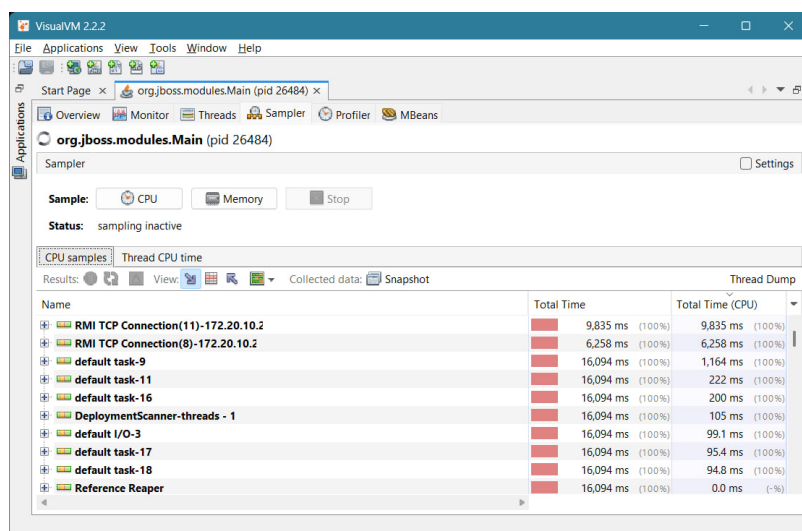


График **TotalArea** принципиально отличается по характеру: это не счётчик, а функция состояния. Линия остаётся горизонтальной всё время, пока радиус не меняется, и совершает мгновенный скачок в момент его изменения

4.4 Определение потока с наибольшим потреблением CPU

Для определения потока использована вкладка **Sampler**, режим CPU. Сбор данных выполнялся в течение 15 секунд при активной работе с приложением. Таблица результатов содержит столбцы Name (имя потока), Total Time (CPU) (чистое процессорное время) и Total Time (общее время наблюдения потока). Сортировка выполнена по процессорному времени по убыванию.



Верхние две строки таблицы занимают потоки **RMI TCP Connection**. Это не потоки прикладной логики: через них VisualVM по протоколу RMI опрашивает виртуальную машину. Их появление в вершине списка - проявление эффекта наблюдателя: средство измерения работает внутри наблюдаемого процесса, потребляет его ресурсы и попадает в собственные измерения.

Среди потоков, действительно выполняющих работу приложения, наибольшее процессорное время потребляет **default task-9**. Это один из рабочих потоков веб-контейнера Undertow, входящего в состав WildFly; такие потоки берут HTTP-запросы из очереди и выполняют всю обработку, включая код JSF, логику проверки попадания точки, обращение к базе данных и обновление MBean-ов.

5 Поиск и устранение проблемы производительности в программе HttpUnit

Исследуемая программа - проект NetBeans: класс `Main` в бесконечном цикле выполняет HTTP-запросы к сервлету `HelloWorld` через встроенный `ServletRunner` и печатает счётчик выполненных запросов.

5.1 Описание

Выявлена **утечка памяти** — неограниченный рост статической коллекции `_errorMessages` в классе `com.meterware.httpunit.javascript.JavaScript`.

Проблема складывается из трёх составляющих, каждая из которых по отдельности безобидна и не является ошибкой.

Класс `HelloWorld` выводит в генерируемую страницу вызов `document.wr(...)` вместо `document.write(...)`. Такого метода не существует, поэтому при разборе каждой полученной страницы выбрасывается `TypeError`.

Класс `Main` вызывает `HttpUnitOptions.setExceptionsThrownOnScriptError(false)`. В этом режиме библиотека не прерывает работу при ошибке скрипта, а сохраняет её текст для последующего чтения приложением.

Сообщение помещается в статический список `_errorMessages`. Статическое поле само является корнем достижимости для сборщика мусора, поэтому попавшие в список строки не могут быть собраны никогда. Методы `clearErrorMessages()` и `getErrorMessages()` в библиотеке предусмотрены, но программа их не вызывает, а её цикл бесконечен.

В результате на каждый выполненный HTTP-запрос в куче навсегда остаётся одно сообщение об ошибке. Текст сообщения одинаков для всех элементов списка:

5.2 Воспроизведение

Последовательность поиска. В проекте сохранены оригинальные исходные тексты и подготовлен запускающий сценарий `find-bug.bat`.

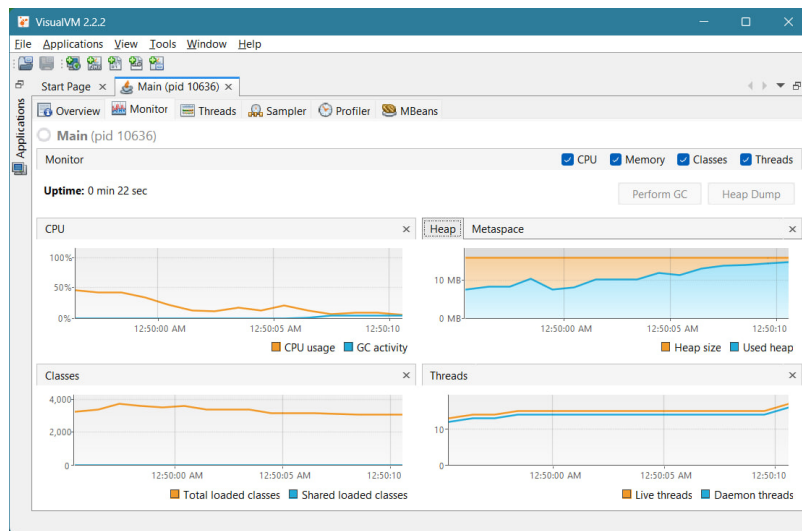
5.2.1 Подготовка и запуск

Программа собирается и запускается из командной строки.

Чтобы утечка проявлялась за минуту, а не за час, сценарий запускает программу с отличием от штатного режима. Используется копия исходного класса `Main`, в которой пауза между запросами уменьшена с 200 мс до нуля: изменён ровно один символ, логика не затронута, изменился только темп.

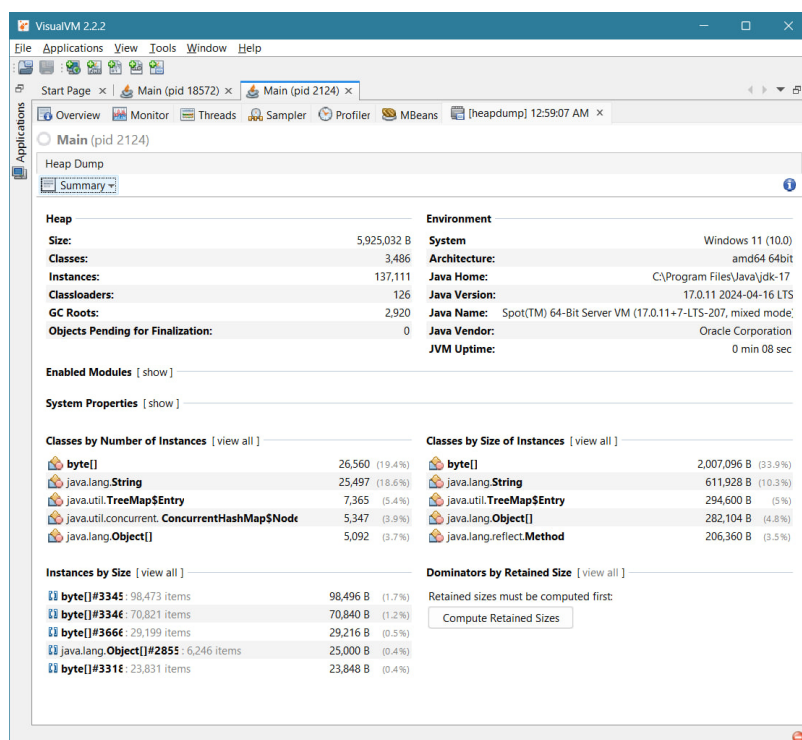
5.2.2 Обнаружение симптома на вкладке Monitor

К процессу подключается `VisualVM`, открывается вкладка `Monitor` и график `Heap`. График принимает пилообразный вид с монотонно растущим основанием: после каждой сборки мусора минимум оказывается выше предыдущего.



```
Count: 10192 [_response = com.meterware.servletunit.ServletUnitHttpResponse@70560769]
Count: 10193 [_response = com.meterware.servletunit.ServletUnitHttpResponse@7f25d78a]
Count: 10194 [_response = com.meterware.servletunit.ServletUnitHttpResponse@4e4bf305]
Count: 10195 [_response = com.meterware.servletunit.ServletUnitHttpResponse@440eba78]
Count: 10196 [_response = com.meterware.servletunit.ServletUnitHttpResponse@4e0e3e48]
Count: 10197 [_response = com.meterware.servletunit.ServletUnitHttpResponse@22741e1d]
Exception in thread "main" java.lang.OutOfMemoryError: Java heap space
    at java.base/java.io.BufferedWriter.<init>(BufferedWriter.java:99)
```

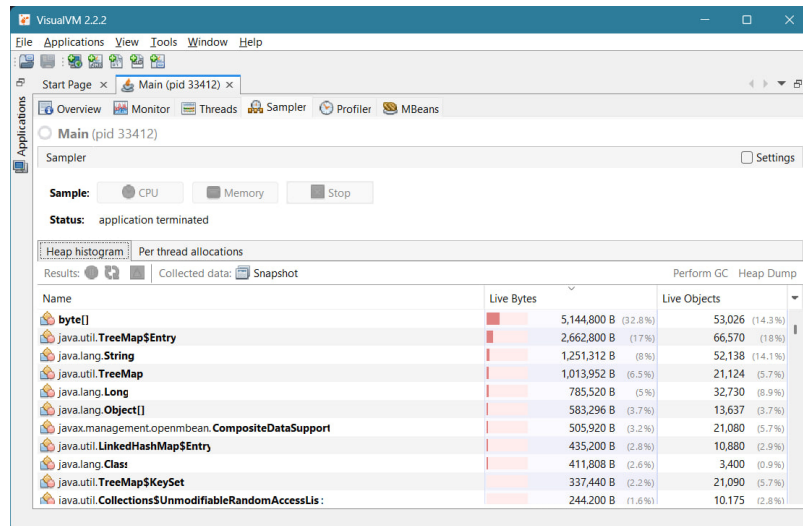
Ключевым диагностическим действием является нажатие кнопки **Perform GC**. Эта кнопка настоятельно рекомендует виртуальной машине выполнить полную сборку. Если после принудительной сборки занятая память возвращается к исходному уровню, значит объекты были обычным мусором; если не возвращается — их кто-то удерживает ссылками, и это утечка. Два вызова Perform GC с интервалом в полминуты показали, что уровень занятой памяти после каждого из них оказывается выше предыдущего.



Проблема существует и относится к классу утечек памяти.

5.2.3 Определение типа утекающих объектов

Открывается вкладка **Sampler**, режим **Memory**, таблица сортируется по столбцу **Live Bytes** по убыванию. Верхние позиции занимают `java.lang.String` и массивы символов, причём столбец **Live Objects** для них растёт с той же скоростью, что и счётчик выполненных запросов в консоли программы, — примерно один объект на запрос.



То есть, утекают строки, по одной на каждый запрос. Совпадение скорости роста со счётчиком запросов связывает утечку непосредственно с обработкой запроса, а не с фоновой активностью.

5.2.4 Подтверждение по исходному коду

Чтение исходных текстов замыкает цепочку рассуждений. В классе `JavaScript` обнаруживается объявление статического списка и его заполнение в обработчике ошибок, а также присутствующие, но нигде не вызываемые методы очистки и чтения. В классе `Main` обнаруживается отключение исключений на ошибках скриптов и отсутствие вызова очистки в бесконечном цикле. В классе `HelloWorld` обнаруживается опечатка в имени метода `JavaScript`.

Каждое из трёх наблюдений по отдельности не является ошибкой, но вместе они дают утечку, что и объясняет, почему проблема не обнаруживается чтением кода одного класса.

5.3 Исправление

В сервлете исправляется опечатка: `document.wr` заменяется на `document.write`. Скрипт становится корректным, исключений не возникает, и в список нечего добавлять.

5.4 Сравнение

В одинаковых условиях запущена та же ускоренная программа, скомпилированная с исправленным сервлетом. Скорость выполнения запросов, нагрузка и параметры виртуальной машины полностью совпадают с первым запуском; отличается только наличие исправления. Это исключает возражение о том, что различие графиков могло быть вызвано разной интенсивностью работы.

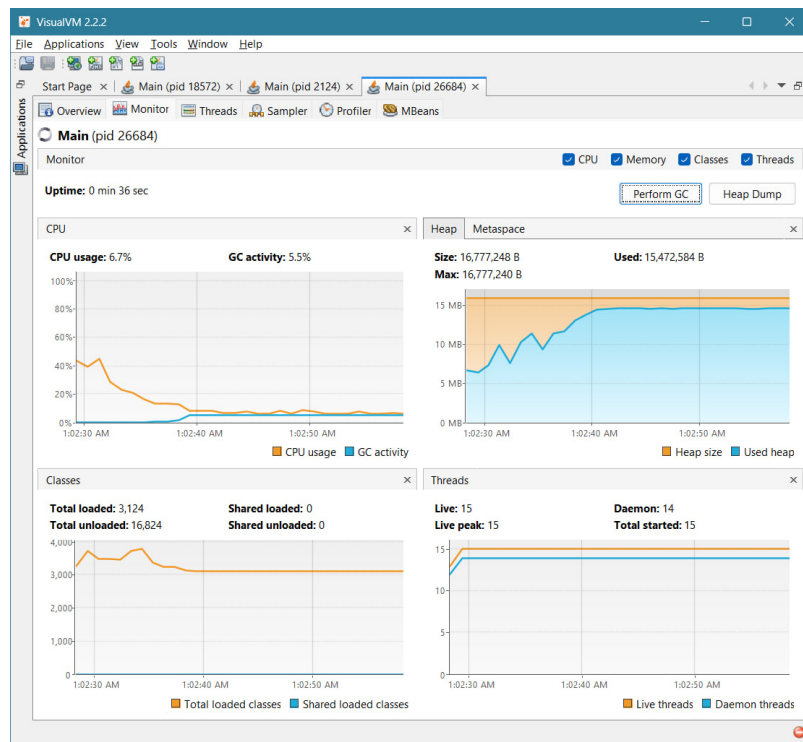


График Heap принимает вид пилы с постоянным основанием: после каждой сборки занятая память возвращается на прежний уровень. Нажатие Perform GC возвращает показание к исходному значению, чего в первом случае не происходило.

6 Выводы

В ходе выполнения лабораторной работы изучены средства мониторинга и профилирования приложений на платформе Java. Разработаны два собственных MBean-класса, практически освоена работа с утилитами JConsole и VisualVM. Освоена методика поиска утечек памяти.